# Promesas y async/await: del callback hell al código legible

**Objetivo:** Recordar el problema de encadenar callbacks y presentar la herramienta que lo resuelve.

## Contenido

- Con callbacks, cada paso que depende del anterior va **dentro** del anterior.
- Tres pasos ya forman una "pirámide" difícil de leer y de corregir: el famoso **Callback Hell**.

Hoy: **promesas** y **async/await** para escribir lo mismo de forma plana y con mejor manejo de errores.

```
setTimeout(() => {
  console.log("1. Pedido recibido");
  setTimeout(() => {
    console.log("2. Pedido
empacado");
    setTimeout(() =>
      console.log("3. Pedido
enviado"), 500);
  }, 500);
}, 500);
```

Consola: 1. Pedido recibido → 2. Pedido empacado → 3. Pedido enviado



# ¿Qué es una promesa?

**Objetivo:** Entender la promesa como un valor que llegará ahora, más tarde o nunca.

## Contenido

Una **promesa** es un objeto que representa el resultado de una tarea asíncrona: red, archivos, bases de datos...

**Caso real:** el mensajero olvidó tu paquete. Un compañero **promete** avisarte cuando sepa algo. Tú **sigues trabajando**. Al final llega una de dos noticias: "lo encontramos" o "no apareció, y este es el motivo".

```
const paquete = new
Promise((resolve) =>
  resolve("Paquete encontrado"));
console.log(paquete);
paquete.then((valor) =>
console.log(valor));
```

Consola: Promise {<fulfilled>: 'Paquete encontrado'} → Paquete encontrado

significa **“cumplida”** en inglés. Es el nombre técnico del estado de una promesa que terminó bien.



# Estados de una promesa

**Objetivo:** Reconocer los tres estados y saber que el resultado final no cambia.

### ⏳ Pendiente (pending)

La tarea aún no termina

### ✅ Cumplida (fulfilled)

Terminó bien y entrega un valor

### ❌ Rechazada (rejected)

Falló y entrega un motivo

Una vez cumplida o rechazada, la promesa queda **establecida** (settled): ya no cambia.

```
const p = new Promise((resolve, reject) => {
  resolve("cumplida");
  reject("rechazada"); // se ignora: ya se estableció
});
p.then((v) => console.log(v)).catch((e) => console.log(e));
```

Consola: cumplida

# 🤔 Pregunta rápida

**Objetivo:** Comprobar la idea de promesa y sus estados.

| 1 | 2 | 3 |
| --- | --- | --- |
| **Si una tarea tarda 2 segundos, ¿en qué estado está su promesa justo después de crearla?** | **Una promesa ya cumplida, ¿puede pasar después a rechazada?** | **¿Qué imprime este código?** |

```
const p = new Promise((resolve, reject) => {
 reject("No");
 resolve("Sí");
});
p.then((v) => console.log(v)).catch((e) => console.log(e));
```

# Cómo se declara una promesa

**Objetivo:** Crear promesas con new Promise, resolve y reject.

## Contenido

- `new Promise((resolve, reject) => { ... })` recibe una **función ejecutora**.
- Si todo sale bien → `resolve(valor)`. Si algo falla → `reject(motivo)`.
- Ojo: la función ejecutora **se ejecuta de inmediato**; lo que espera es el resolve.

Tip: envuelve la promesa en una función para reutilizarla: `const esperar = (ms, valor) => new Promise((resolve) => setTimeout(() => resolve(valor), ms));`

```
const pedido = new Promise((resolve, reject) => {
  console.log("Buscando paquete...");
  setTimeout(() =>
    resolve("Paquete en camino"), 2000);
});
console.log("Sigo trabajando");
```

Consola: Buscando paquete... → Sigo trabajando

# Cómo se consume: .then(), .catch() y .finally()

**Objetivo:** Reaccionar al éxito, al error y al cierre de una promesa.

## Explicación

- `.then(fn)`: corre fn con el **valor** si la promesa se cumple.
- `.catch(fn)`: corre fn con el **motivo** si se rechaza.
- `.finally(fn)`: corre fn en ambos casos (no recibe valor).
- Con `exito = false`, el control salta directo al `.catch()`.

```
const exito = false;
const pedido = new Promise((resolve, reject) => {
  if (exito) {
    resolve("Paquete entregado");
  } else {
    reject("Paquete perdido");
  }
});
pedido
  .then((res) => console.log("OK:", res))
  .catch((error) => console.log("ERROR:", error))
  .finally(() => console.log("Fin del seguimiento"));
```

Consola: ERROR: Paquete perdido → Fin del seguimiento

# Encadenar .then()

**Objetivo:** Pasar el resultado de un paso al siguiente sin anidar.

## Contenido

- Lo que **retorna** un `.then()` llega como parámetro al siguiente `.then()`.
- Si retornas una promesa, el siguiente `.then()` **espera** a que se cumpla.
- Con llaves `{ }` y sin `return`, el siguiente recibe `undefined`.

```
// Devuelve una promesa que se cumple con "valor" después de "ms" milisegundos
const esperar = (ms, valor) => new Promise((resolve) => {
  setTimeout(() => resolve(valor), ms);
});
```

```
esperar(500, 10)
  .then((n) => n * 2)
  .then((n) => console.log("Resultado:", n));

esperar(600, 10)
  .then((n) => { n * 2; })
  .then((n) => console.log("Sin return:", n));
```

Consola: Resultado: 20 → Sin return: undefined

`Resultado` sale primero porque esa promesa tarda 500 ms y la otra 600 ms.

# 🤔 Pregunta rápida

**Objetivo:** Verificar declaración, consumo y encadenamiento.

## 1

### ¿Qué imprime y en qué orden?

```
const p = new Promise((resolve) => {
  console.log("1");
  resolve("2");
});
p.then((v) => console.log(v));
console.log("3");
```

## 2

### ¿En qué casos se ejecuta .finally()?

## 3

### ¿Qué imprime?

```
esperar(100, 5)
 .then((n) => { n + 1; })
 .then((n) => console.log(n));
```

# Las promesas no bloquean

**Objetivo:** Predecir el orden de ejecución con promesas.

## Contenido

- El código dentro de `.then()` es **asíncrono** y **no bloqueante**: el programa sigue mientras espera.
- Aunque la promesa ya esté cumplida, `.then()` **nunca** corre de inmediato: primero termina el código síncrono.

```
console.log("Inicio");
esperar(2000, "Promesa terminada")
  .then((res) => console.log(res));
new Promise((resolve) =>
  resolve("Ya estaba lista"))
  .then((v) => console.log(v));
console.log("Fin");
```

Consola: Inicio → Fin → Ya estaba lista → Promesa terminada

# Promise.race(): la primera que termine

**Objetivo:** Quedarse con la primera promesa que se establezca, sea éxito o error.

## Contenido

- Recibe un array de promesas y adopta el resultado de la **primera que termine**.
- Si la primera se **rechaza**, race también se rechaza.
- Uso típico: el servidor más rápido, o un **tiempo límite**.

```
// Devuelve una promesa que se cumple con "valor" después de "ms" milisegundos
const esperar = (ms, valor) => new Promise((resolve) => {
  setTimeout(() => resolve(valor), ms);
});
```

```
const fallar = (ms, motivo) =>
  new Promise((resolve, reject) =>
    setTimeout(() => reject(motivo), ms));

Promise.race([
  esperar(1000, "Servidor A"),
  fallar(500, "Servidor B caído")
])
  .then((res) => console.log("Ganó:", res))
  .catch((error) =>
    console.log("El primero falló:", error));
```

Consola: El primero falló: Servidor B caído

# Promise.all(): necesito todas

**Objetivo:** Esperar varias tareas y recibir todos sus resultados juntos.

## Contenido

- Se cumple con un **array de resultados** cuando **todas** se cumplen.
- El array respeta el **orden en que las pasaste**, no el orden de llegada.
- Los resultados llegan en este orden: **P3, P1, P2**. Sin embargo, la consola muestra:

```
['P1', 'P2', 'P3']
```

- Si **una** falla, all se rechaza con el motivo de la primera que falle.
- `Promise.all` es **pesimista**: para cumplirse espera a todas, pero para fallar le basta la primera que falle.

```
Promise.all([
  esperar(1000, "P1"),
  esperar(1500, "P2"),
  esperar(500, "P3")
]).then((resultados) => console.log(resultados));

Promise.all([
  esperar(1000, "P1"),
  fallar(300, "P2 falló")
]).catch((error) => console.log("Falló:", error));
```

Consola: Falló: P2 falló → ['P1', 'P2', 'P3']

- **Primer** `Promise.all`**, termina a los 1500 ms.** Para cumplirse necesita que **todas** sus promesas se cumplan, así que espera a la más lenta:
- **Segundo** `Promise.all`**, termina a los 300 ms.** Le basta **un solo rechazo** para rechazarse. `fallar(300, ...)` se rechaza a los 300 ms y `Promise.all` falla en ese momento, sin esperar a P1

# 🤔 Pregunta rápida

**Objetivo:** Repasar orden de ejecución, race y all.

## 1

**¿Qué imprime?**

```
Promise.all([
  esperar(300, "A"),
  esperar(100, "B")
]).then((r) => console.log(r));
```

## 2

**En Promise.race, si la primera promesa en terminar se rechaza, ¿se ejecuta .then() o .catch()?**

## 3

**¿Por qué console.log("Fin") sale antes que el .then() de una promesa que ya estaba cumplida?**

# Promise.any(): el primer éxito

**Objetivo:** Quedarse con la primera promesa que se cumpla, ignorando los rechazos.

## Contenido

- Se cumple con el **primer éxito** (ES2021).
- Solo se rechaza si **todas** fallan: entrega un `AggregateError` cuya propiedad `.errors` es un array con todos los motivos.

```
Promise.any([
  fallar(800, "E1"),
  esperar(600, "OK 2"),
  fallar(1000, "E3")
]).then((valor) =>
  console.log("Primera exitosa:", valor));

Promise.any([
  fallar(800, "E1"),
  fallar(300, "E2")
]).catch((error) => console.log(error.errors));
```

Consola: Primera exitosa: OK 2 → ['E1', 'E2']

# async: funciones que devuelven promesas

**Objetivo:** Saber qué cambia al declarar una función con async.

## Contenido

- `async` declara una función asíncrona.
- Una función async **siempre devuelve una promesa**: lo que retornes queda envuelto en ella.
- Por eso no necesitas escribir `new Promise` para devolver un valor.

```
async function saludar() {
  return "Hola, mundo";
}
console.log(saludar());
saludar().then((v) => console.log(v));
```

Consola: Promise {<fulfilled>: 'Hola, mundo'} → Hola, mundo

# await: esperar sin bloquear

**Objetivo:** Obtener el valor de una promesa con await y entender qué se pausa.

## Contenido

- `await` solo se usa dentro de una función `async` (salvo en módulos, ver más adelante).
- Pausa **esa función** hasta que la promesa se cumpla y entrega su valor directamente.
- El resto del programa **sigue**: no se bloquea.

```
// Devuelve una promesa que se cumple con "valor" después de "ms" milisegundos
const esperar = (ms, valor) => new Promise((resolve) => {
  setTimeout(() => resolve(valor), ms);
});
```

```
async function ejemplo() {
  console.log("A");
  const r = await esperar(1000, "B");
  console.log(r);
}
ejemplo();
console.log("C");
```

Consola: A → C → B

# 🤔 Pregunta rápida

**Objetivo:** Comprobar any, async y await.

## 1

**¿Qué imprime?**

```
async function f() { return 5; }
console.log(f());
```

## 2

**¿Qué imprime y en qué orden?**

```
async function g() {
  console.log("1");
  await esperar(500);
  console.log("2");
}
g();
console.log("3");
```

## 3

**Si la promesa más rápida falla, ¿qué hace Promise.race y qué hace Promise.any?**

# Errores con async/await

**Objetivo:** Saber cuándo una función async termina en .catch() y cuándo no.

## Explicación

- Si haces `await` a una promesa **rechazada**, la función se detiene y su promesa se **rechaza** → lo atrapas con `.catch()` al llamarla.
- Retornar un objeto como `{ error: true }` **no** rechaza: la promesa se **cumple** y entra al `.then()`.
- Ejemplo de los dados: hasta 10 tiradas; si sale un 6 se interrumpe. Las dos salidas llegan al `.then()`.

```
async function entregar() {
  await fallar(500, "Dirección inválida");
  console.log("Esto no se imprime");
}
entregar().catch((e) =>
  console.log("Capturado:", e));

const tirarDados = async (veces) => {
  const numeros = [];
  for (let i = 0; i < veces; i++) {
    const n = 1 + Math.floor(Math.random() * 6);
    numeros.push(n);
    if (n === 6) {
      return { error: true,
        message: "Se ha sacado un 6" };
    }
  }
  return { error: false, value: numeros };
};
tirarDados(10).then((res) =>
  console.log("then:", res));
```

Consola: then: {error: true, message: 'Se ha sacado un 6'} (o then: {error: false, value: [...]}; es aleatorio) → a los 500 ms: Capturado: Dirección inválida

# fetch: pedir datos con async/await

**Objetivo:** Leer un JSON con fetch y combinar .then() con await.

## Contenido

- `fetch(url)` devuelve una promesa con la respuesta. `response.json()` y `response.text()` **también** devuelven promesas: por eso hay dos esperas.
- `json()` entrega un objeto; `text()`, un string.
- Se puede mezclar: `.then()` para la primera promesa y `await` para el resultado final.
- fetch necesita un servidor local (por ejemplo, Live Server); abriendo el HTML con doble clic falla.

```
async function cargar() {
  const response =
    await fetch("json/data.json");
  const data = await response.json();
  console.log(data);
}
cargar();
```

```
async function obtenerTexto() {
  const texto = await fetch("json/data.json")
    .then((r) => r.text());
  console.log(texto);
}

obtenerTexto();
```

# Top-level await

**Objetivo:** Saber dónde se puede usar await fuera de una función async.

## Contenido

- **Top-level await:** usar `await` en el nivel superior del archivo. Solo funciona en **módulos**: `<script type="module">`.
- En un `<script>` normal da `SyntaxError: await is only valid in async functions and the top level bodies of modules`.
- Alternativa clásica: una función async autoejecutable.

```
<script type="module" src="js/main.js"></script>

const data = await fetch("json/data.json")
  .then((r) => r.json());
console.log(data.informacion_personal);
```

Consola: JavaScript

```
// Permite usar await en un script normal, sin type="module".
(async () => {
  const data = await fetch("json/data.json")
    .then((r) => r.json());
  console.log(data.informacion_personal);
})();
```

# 🤔 Pregunta rápida

**Objetivo:** Verificar manejo de errores, fetch y top-level await.

| 1 | 2 | 3 |
| --- | --- | --- |
| Si una función async hace return { error: true }, ¿se ejecuta su .then() o su .catch()? | ¿Qué devuelve response.json(): un objeto o una promesa? | ¿Por qué falla un await suelto en js/main.js cargado con <script src="js/main.js">? |

# Chuleta: ¿qué necesito y qué uso?

**Objetivo:** Conectar problemas típicos con la herramienta adecuada antes de practicar.

| Necesito... | Uso... |
| --- | --- |
| Representar una tarea que termina después | `new Promise((resolve, reject) => ...)` |
| Reaccionar si sale bien / si falla / siempre | `.then()` / `.catch()` / `.finally()` |
| Hacer un paso después de otro | `.then()` con `return` o varios `await` |
| Esperar varias y usar **todas** | `Promise.all([...])` |
| La **primera** que termine (éxito o error) | `Promise.race([...])` |
| El **primer éxito** | `Promise.any([...])` |
| Poner un tiempo límite | `Promise.race([tarea, limite])` |
| Escribir código asíncrono como una secuencia | `async function` + `await` |
| Atrapar el error de una función async | `miFuncion().catch(...)` |
| Pedir un JSON | `fetch()` + `response.json()` |
| Usar await fuera de funciones | `<script type="module">` |

# Práctica: orden de trabajo y reglas de juego

**Objetivo:** Conocer la ruta de la práctica y las reglas comunes a todos los ejercicios.

**Orden:** ✅ Checkpoints → 🏫 Guiados → 🏠 Para la casa → 🔥 Reto integrador

## Reglas de juego:

Trabaja con `index.html` + `js/main.js`; el script va justo antes de `</body>`.

No modifiques los datos ni el código base, salvo donde el ejercicio lo pida.

Comenta cada requisito: `// R1`, `// R2`...

`const` por defecto; `let` solo si reasignas; nunca `var`.

Compara con `===` y `!==`.

Sin librerías externas.

Verifica cada resultado en la consola (F12).

# ✅ Checkpoint 1 · Cafetería

**Objetivo:** Combinar declaración de promesas, then/catch/finally y orden de ejecución.

## Retos

1. Pide un "tinto": muestra el resultado con `.then()` y el error con `.catch()`.
2. Agrega `.finally()` que imprima "Siguiente cliente".
3. Pide un "Tinto" (con mayúscula). ¿Qué pasa y por qué?
4. Al final del archivo escribe `console.log("Cierra la caja")`. **Explica por qué** "Pedido recibido" sale antes que "Cierra la caja", pero "Tinto listo" sale después.

## Código base

```
const prepararCafe = (tipo) =>
 new Promise((resolve, reject) => {
 console.log("Pedido recibido:", tipo);
 setTimeout(() => {
 if (tipo === "tinto") {
 resolve("Tinto listo");
 } else {
 reject("No hay " + tipo);
 }
 }, 1000);
 });
console.log("Abre la cafetería");
```

# ✅ Checkpoint 2 · Notas de la plataforma

**Objetivo:** Combinar encadenamiento de .then(), conversión de tipos y alcance/orden.

## Retos

1. Encadena dos `.then()`: el primero suma un bono de 0.5 y lo retorna; el segundo lo imprime. ¿Qué sale y por qué?
2. Corrígelo para obtener 4.
3. Agrega un tercer `.then()` que imprima "Aprobado" si la nota es >= 3. Si dice "Reprobado", **explica por qué** y corrígelo.
4. Dentro del primer `.then()` guarda la nota en `notaFinal`. Al final del archivo imprime `notaFinal`. **Explica por qué** sale `undefined`.

## Código base

```
const pedirNota = () =>
 new Promise((resolve) => {
 setTimeout(() =>
 resolve("3.5"), 500);
 // llega como texto
 });
let notaFinal;
```

# ✅ Checkpoint 3 · Comparador de envíos

**Objetivo:** Combinar Promise.race, Promise.any, Promise.all y destructuración.

## Retos

1. Usa `Promise.race(pedirCotizaciones())`. ¿Por qué entra al `.catch()`?
2. Usa `Promise.any()` y, con destructuración, imprime "Cóndor cobra $12000".
3. Usa `Promise.all()`. ¿Qué pasa si 2 de 3 respondieron bien?
4. Llama `Promise.all` con un array nuevo donde Rayo cobra 7000. **Explica por qué** el resultado sale en el orden Rayo, Tortuga, Cóndor y no por tiempo de llegada.

## Código base

```
const cotizar = (empresa, ms, precio) =>
 new Promise((resolve, reject) => {
 setTimeout(() => {
 if (precio > 0) {
 resolve({ empresa, precio });
 } else {
 reject(empresa + ": sin cobertura");
 }
 }, ms);
 });

const pedirCotizaciones = () => [
 cotizar("Rayo", 300, 0),
 cotizar("Tortuga", 900, 8000),
 cotizar("Cóndor", 600, 12000),
];
```

# ✅ Checkpoint 4 · Inventario de la tienda

**Objetivo:** Combinar async, await y Promise.all; comparar espera en serie y en paralelo.

## Retos

1. Crea `async function revisar()` que espere `consultarStock("mouse", 1000)` e imprima "mouse: 3 unidades".
2. Llámala con `console.log(revisar())`. ¿Qué se imprime primero y por qué?
3. Quita el `await`. ¿Qué imprime ahora y por qué?
4. Consulta "teclado" y "monitor" (1000 ms cada uno): primero con dos `await` seguidos y luego con `Promise.all`. ¿Cuál termina antes del aviso de 1.5 s? **Explica por qué.**

## Código base

```
const consultarStock = (producto, ms) =>
 new Promise((resolve) => {
 setTimeout(() =>
 resolve({ producto, unidades: 3 }), ms);
 });

setTimeout(() =>
 console.log("--- pasaron 1.5 s ---"), 1500);
```

# 🏫 Ejercicio guiado 1 · Factura del restaurante La Esquina

**Objetivo:** Resolver un caso real con Promise.all, encadenamiento, catch/finally y conversión de tipos.

## Requisitos

- **R1** Crea `facturar(ids)`: pide todos los platos a la vez con `Promise.all`.
- **R2** Lista cada plato: - Ajiaco: $28000.
- **R3** Suma el subtotal (cuidado con el precio en texto) y retórnalo al siguiente `.then()`.
- **R4** Calcula impoconsumo 8 % (redondeado) y total.
- **R5** `.catch()`: "No se pudo facturar: <motivo>".
- **R6** `.finally()`: "Gracias por visitarnos".
- **R7** Cuando termine `facturar([1, 2, 3])`, llama `facturar([1, "3"])`.

**Salida esperada:** 3 platos → Subtotal: $67000 → Impoconsumo (8%): $5360 → Total: $72360 → Gracias por visitarnos → No se pudo facturar: Plato 3 no existe → Gracias por visitarnos

## Código base

```
const carta = [
 { id: 1, plato: "Bandeja paisa",
 precio: 32000 },
 { id: 2, plato: "Ajiaco",
 precio: "28000" },
 { id: 3, plato: "Limonada",
 precio: 7000 },
];
const IMPOCONSUMO = 0.08;
// pedirPlato(id): promesa que busca
// con === en 300 ms
// y rechaza con "Plato <id> no existe"
```

# 🏫 Ejercicio guiado 2 · Inscripciones en la Academia CódigoYa

**Objetivo:** Resolver un caso real con async/await, rechazos, spread y comparación estricta.

## Requisitos

- **R1** `async function inscribir(documento, codigo)`: espera al estudiante y luego al curso.
- **R2** Con `await validar(...)`, rechaza si el curso no tiene cupos.
- **R3** Rechaza si el saldo no alcanza.
- **R4** Crea la inscripción con spread y el saldo descontado; muestra que el original no cambió.
- **R5** Atrapa errores con `.catch()` al llamar la función.
- **R6** Corre 4 casos en orden: ("1020","JS1"), ("3344","JS1"), ("1020","PY1"), (1020,"JS1").

**Salida esperada:** ✔️ Laura inscrita... Saldo restante: $30000 / Saldo en el registro original: $150000 / ✖️ Saldo insuficiente de Andrés / ✖️ Sin cupos en Python Inicial / ✖️ Estudiante 1020 no registrado

## Código base

```
const estudiantes = [
 { documento: "1020", nombre: "Laura",
 saldo: 150000 },
 { documento: "3344", nombre: "Andrés",
 saldo: 40000 },
];
const cursos = [
 { codigo: "JS1",
 nombre: "JavaScript Básico",
 cupos: 2, precio: 120000 },
 { codigo: "PY1",
 nombre: "Python Inicial",
 cupos: 0, precio: 100000 },
];
// buscarEstudiante(documento),
// buscarCurso(codigo): promesas con ===
const validar = (condicion, motivo) =>
 new Promise((resolve, reject) => {
 if (condicion) { resolve(true); }
 else { reject(motivo); }
 });
```

# 🏠 Para la casa

**Objetivo:** Practicar de forma autónoma con casos de dificultad similar a los guiados.

| Ejercicio | Temas que combina |
|---|---|
| **1. Seguimiento de domicilio · RapiMandados** | new Promise, .then() encadenado con return, .catch(), .finally(), spread, orden de ejecución |
| **2. Taquilla del Cine Las Palmas** | async/await, Promise.all, destructuración, NaN en precios con coma, .catch() en la llamada |
| **3. Conversor de divisas CambiaYa** | Promise.any, Promise.race con tiempo límite, AggregateError, conversión con * y + |

Enunciados completos, código base y salida esperada: en el README.

# 🔥 Reto integrador · Checkout de TecnoBarrio

**Objetivo:** Integrar promesas y async/await en un programa completo con forma de evaluación.

Una tienda online procesa el carrito de Mateo: consulta productos, elige bodega, cobra con tiempo límite y confirma el pedido.

| Req. | Qué debe hacer | Concepto que evalúa |
|---|---|---|
| **R1** | `procesarPago(total, cupo)`: promesa que en 700 ms aprueba o rechaza "Fondos insuficientes" | Declarar promesas |
| **R2** | `async function checkout(carrito, cupo)` consulta todos los productos a la vez | Promise.all + await |
| **R3** | Subtotal por línea y total con envío (el precio y el envío llegan como texto) | Conversión de tipos |
| **R4** | Usa la primera bodega con inventario | Promise.any |
| **R5** | El pago compite contra `limiteDeTiempo(1000)` | Promise.race |
| **R6** | Arma el pedido con spread e imprímelo con destructuración | Objetos, spread, destructuración |
| **R7** | 3 casos en orden, cada uno con `.catch()` y `.finally()` | Manejo de errores |
| **R8** | Imprime mensajes antes y después de llamar `main()` y explica el orden en un comentario | No bloqueo / orden de ejecución |

# 🔥 Reto integrador · Entregable y rúbrica

**Objetivo:** Conocer qué se entrega y cómo se califica.

**Entregable:** carpeta `reto-apellido-nombre` con `index.html` y `js/main.js`, código comentado `// R1…R8` y una captura de la consola con los 3 casos.

**Salida esperada (resumen):** Caso 1 → Total: $198000, pago aprobado desde Bodega Norte · Caso 2 → Fondos insuficientes · Caso 3 → Producto 9 no existe · cada caso cierra con Sesión de compra cerrada.

| Criterio | % |
|---|---|
| R1 · Promesa de pago con resolve/reject | 10 % |
| R2 · Promise.all + await | 15 % |
| R3 · Cálculos correctos con conversión | 15 % |
| R4 · Promise.any | 10 % |
| R5 · Promise.race con límite | 10 % |
| R6 · Pedido con spread y destructuración | 10 % |
| R7 · catch/finally en los 3 casos | 15 % |
| R8 · Orden de ejecución explicado | 5 % |
| Buenas prácticas (const/let, ===, // R, sin var) | 10 % |

# Resumen y lista "Antes de entregar"

**Objetivo:** Cerrar con las ideas clave y una revisión final del trabajo.

**Ideas clave:** una promesa es un valor futuro con 3 estados · then/catch/finally reaccionan a su resultado · all, race y any coordinan varias · async/await hace que el código asíncrono se lea como una secuencia, sin bloquear.

## Antes de entregar ✔️

- El script está antes de `</body>` y la consola no muestra errores.
- Cada requisito tiene su comentario `// R1`, `// R2`...
- Uso `const`/`let`, nunca `var`; comparo con `===`/`!==`.
- Cada `.then()` que debe pasar un valor tiene `return`.
- Toda promesa que puede fallar tiene su `.catch()`.
- Convertí con `Number()` los datos que llegan como texto y revisé que no haya `NaN`.
- Cada `await` está dentro de una función `async`.
- La salida de la consola coincide con la esperada.